

Reinforcement Learning (Q-Learning)

Fundamentals · Hands-on examples · Python Implementation for Game

CSE4020 — Capstone Design | Dr. Oybek Eraliev

Table of Contents

What we will cover in this lecture

01 Types of RL Models

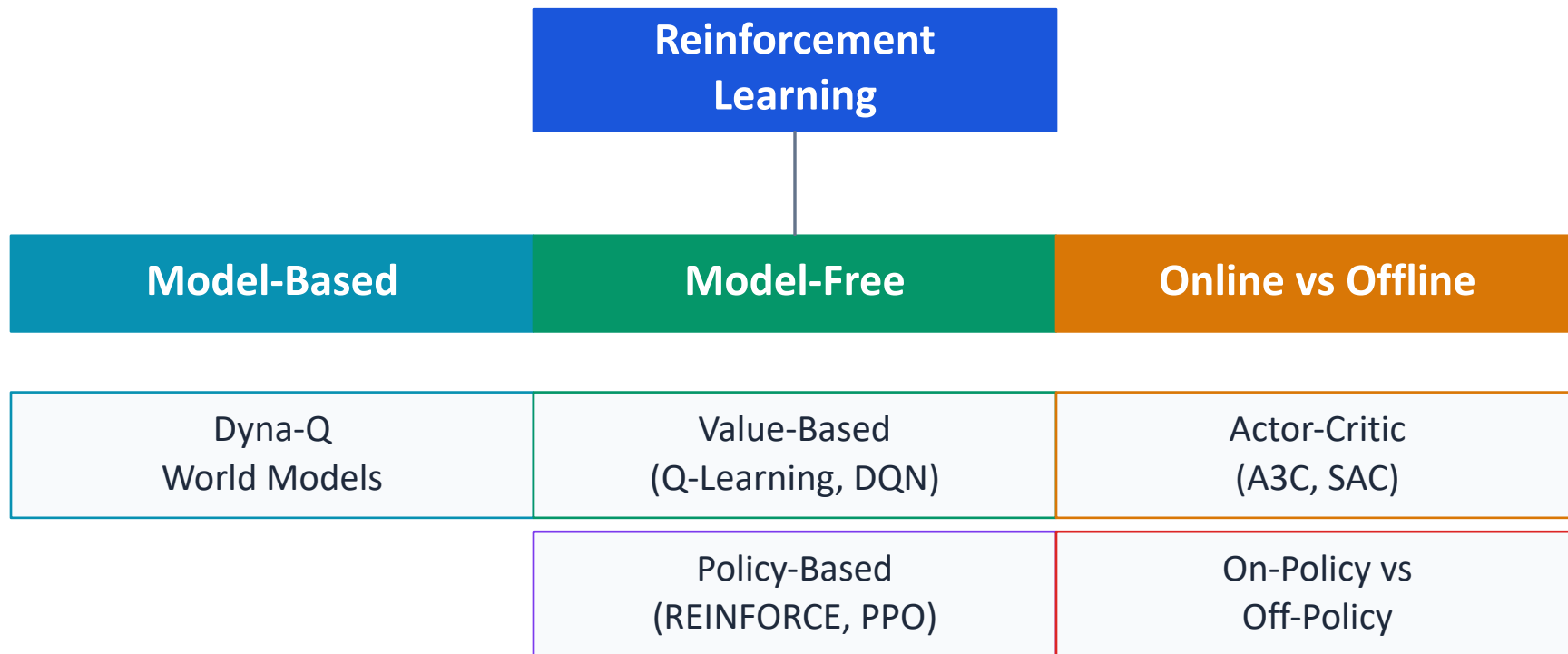
02 Q-Learning Algorithm

03 Q-Learning explanation in maze problem

04 Q-Learning Implementation to Tic-Tac-Toe game using Python

Types of Reinforcement Learning Models

Overview of RL Taxonomy



Model-Based Reinforcement Learning

RL Model Types

The agent builds an internal model of the environment (transition & reward functions), then uses it to plan actions.



Advantages

- Sample efficient — learns with fewer interactions
- Can plan ahead using mental simulation
- Allows offline planning before acting
- Easier to transfer knowledge to new tasks



Disadvantages

- Model errors can mislead the agent
- Building accurate models is hard
- High computational overhead for planning
- May fail in highly stochastic environments

Examples: Dyna-Q (Sutton 1990), AlphaZero (Monte Carlo Tree Search), World Models (Ha & Schmidhuber 2018), MuZero

Model-Free Reinforcement Learning

RL Model Types

The agent learns directly from interactions with the environment — without building an explicit model. Most widely used RL approaches are model-free.

Value-Based

Learn a value function; derive policy from values. Agent picks action with highest Q-value.

Examples: Q-Learning, SARSA, DQN

Policy-Based

Directly optimize the policy π without estimating a value function. Works well in continuous action spaces.

Examples: REINFORCE, TRPO, PPO

Actor-Critic

Hybrid approach: Actor updates the policy, Critic evaluates it using value estimates. Best of both worlds.

Examples: A2C, A3C, SAC, TD3

Model-Free is the dominant paradigm in modern deep RL

Value-Based RL: Q-Learning

Deep Dive into Value-Based Methods

Q-Learning is an off-policy, model-free algorithm that learns the optimal action-value function $Q^*(s,a)$ directly.

Q-Learning Update Rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

α

Learning rate ($0 < \alpha \leq 1$) — how much to update Q on each step

r

Immediate reward received after taking action a in state s

γ

Discount factor — importance of future rewards

$\max_{a'} Q(s', a')$

Best possible future value from next state s' (greedy step)

Value-Based RL: Q-Learning

Q-Learning in maze



S = Start (cell 0,0) $\rightarrow$ state s_0

G = Goal (cell 2,2) $\rightarrow$ state s_8

W = Wall (cell 0,2 and cell 1,1) $\rightarrow$ blocked, cannot enter

Cells are numbered 0–8 left-to-right, top-to-bottom

Value-Based RL: Q-Learning

Q-Learning in maze



1. Agent

The robot navigating the maze. It makes decisions, takes actions, and learns from the results.

2. Environment

The 3×3 maze itself. It responds to the agent's actions and returns new states and rewards.

3. State (s)

A snapshot of where the agent currently is. We have 7 valid states (excluding 2 walls):

Value-Based RL: Q-Learning

Q-Learning in maze



State	Cell	Position
S0	0	(row 0, col 0) — START
S1	1	(row 0, col 1)
S2	3	(row 1, col 0)
S3	5	(row 1, col 2)
S4	6	(row 2, col 0)
S5	7	(row 2, col 1)
S6	8	(row 2, col 2) — GOAL

Value-Based RL: Q-Learning

Q-Learning in maze



4. Action (a)

At each state, the agent can attempt: **Up, Down, Left, Right**. Invalid moves (wall or out-of-bounds) keep the agent in place.

5. Reward (R)

Feedback signal after each action:

- **Reach Goal** $\rightarrow +10$
- **Hit wall / boundary** $\rightarrow -1$ (penalized for wasted moves)
- **Valid move** $\rightarrow -0.1$ (small cost to encourage efficiency)

Value-Based RL: Q-Learning

Q-Learning in maze



4. Action (a)

At each state, the agent can attempt: **Up, Down, Left, Right**. Invalid moves (wall or out-of-bounds) keep the agent in place.

5. Reward (R)

Feedback signal after each action:

- **Reach Goal** $\rightarrow +10$
- **Hit wall / boundary** $\rightarrow -1$ (penalized for wasted moves)
- **Valid move** $\rightarrow -0.1$ (small cost to encourage efficiency)

Value-Based RL: Q-Learning

Q-Learning in maze



6. Policy (π)

The agent's decision rule: which action to take in each state. Q-learning learns the **optimal policy** π^* by building a Q-table.

7. Q-Value: $Q(s, a)$

The expected total future reward for taking action **a** in state **s** and then following the optimal policy afterward. This is what we build and update by hand.

Value-Based RL: Q-Learning

Q-Learning in maze

8. The Bellman Equation (Q-learning update rule)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- **α (alpha)** = Learning rate = **0.5** (how much to update — 0 = don't learn, 1 = overwrite)
- **γ (gamma)** = Discount factor = **0.9** (how much we value future rewards)
- **R** = Immediate reward received
- **s'** = Next state after taking action **a**
- **$\max Q(s', a')$** = Best Q-value possible from the next state

9. Exploration vs. Exploitation (ϵ -greedy)

- With probability **$\epsilon = 0.3$** $\rightarrow$ explore (pick random action)
- With probability **$1-\epsilon = 0.7$** $\rightarrow$ exploit (pick highest Q-value action)

Value-Based RL: Q-Learning

Hand Calculation — Step by Step

Initial Q-Table (all zeros)

Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right
S0 (Start)	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0
S4	0	0	0	0
S5	0	0	0	0
S6 (Goal)	0	0	0	0

Value-Based RL: Q-Learning

Hand Calculation — Step by Step

Initial Q-Table (all zeros) Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right
S0 (Start)	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0
S4	0	0	0	0
S5	0	0	0	0
S6 (Goal)	0	0	0	0



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

ϵ -greedy policy

$$\pi(a|s) = \begin{cases} \text{random action from } \mathcal{A}(s) & \text{with probability } \epsilon \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon \end{cases}$$

Where:

- $\pi(a|s)$ = the policy — probability of choosing action **a** in state **s**
- ϵ = exploration rate (we used 0.3 in our example)
- $\mathcal{A}(s)$ = set of valid actions from state **s**
- $\arg \max Q(s, a)$ = the action with the highest Q-value in state **s**



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

ϵ -greedy policy

Concrete example in our maze at s_0

Suppose after some training the Q-table at s_0 looks like this:

$Q(s_0, \uparrow) = -0.80 \leftarrow$ blocked/bad

$Q(s_0, \downarrow) = +3.37 \leftarrow$ best!

$Q(s_0, \leftarrow) = -0.80 \leftarrow$ blocked/bad

$Q(s_0, \rightarrow) = -0.05 \leftarrow$ valid but not great



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

ϵ -greedy policy

Case 1: random number = 0.15 $\rightarrow$ 0.15 < ϵ (0.3) $\rightarrow$ EXPLORE

Agent ignores the Q-table completely

Picks randomly from valid actions: $\{\downarrow, \rightarrow\}$

Maybe picks $\rightarrow$ even though $Q(s_0, \rightarrow) = -0.05$



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

ϵ -greedy policy

Case 2: random number = 0.72 $\rightarrow 0.72 \geq \epsilon$ (0.3) $\rightarrow$ EXPLOIT

Agent scans Q-table:

$\operatorname{argmax} Q(s_0, a) = \downarrow$ because +3.37 is highest

Agent picks $\downarrow$ deterministically



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

ϵ -greedy policy

Note: The agent does not choose actions randomly, nor does it always pick the best — it uses ϵ -greedy: exploit the Q-table with probability $1-\epsilon$, and explore randomly with probability ϵ , balancing between using current knowledge and discovering something better.



Value-Based RL: Q-Learning

Hand Calculation — Step by Step

Q-Table After Episode 1

Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right
S0 (Start)	0	-0.05	0	0
S1	0	0	0	0
S2	0	-0.05	0	0
S3	0	0	0	0
S4	0	0	0	-0.05
S5	0	0	0	5.0 ← reward propagated!
S6 (Goal)	0	0	0	0

Value-Based RL: Q-Learning

Hand Calculation — Step by Step

Q-Table After Full Training

Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right
S0 (Start)	-0.800	3.374	-0.800	-0.050
S1	-0.800	-0.800	-0.045	-0.800
S2	-0.050	3.86	-0.800	-0.800
S3	-0.800	0	-0.800	-0.800
S4	-0.050	-0.800	-0.800	4.400
S5	-0.800	-0.800	-0.050	5.000
S6 (Goal)	0	0	0	0

Value-Based RL: Q-Learning

Hand Calculation — Step by Step

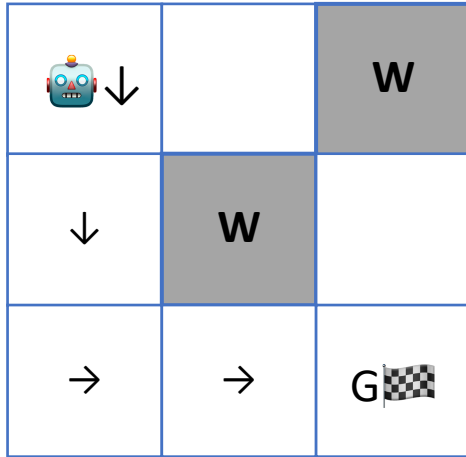
Q-Table After Full Training

Parameters: $\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.3$

State	↑ Up	↓ Down	← Left	→ Right	Optimal Action
S0 (Start)	-0.800	3.374	-0.800	-0.050	↓ Down
S1	-0.800	-0.800	-0.045	-0.800	← Left (back to s_0)
S2	-0.050	3.86	-0.800	-0.800	↓ Down
S3	-0.800	0	-0.800	-0.800	↓ Down → s_5
S4	-0.050	-0.800	-0.800	4.400	→ Right
S5	-0.800	-0.800	-0.050	5.000	→ Right
S6 (Goal)	0	0	0	0	— terminal —

Value-Based RL: Q-Learning

Hand Calculation — Step by Step



The Optimal Policy (after full training)

State	Best Action	Leads to
S0 (Start)	↓ Down	S2
S2	↓ Down	S4
S4	→ Right	S5
S5	→ Right	S6 ✓ Goal

Value-Based RL: Q-Learning

Implementation to Tic-Tac-Toe game

Thank You!

Introduction to Reinforcement Learning

Coming Up:

- Deep RL: DQN
- Train your first PPO agent

Questions?

Dr. Oybek Eraliev
CSE4020 — Capstone Design